

Welcome to the MyFantasyLeague.com Developers Program!

We want to thank you for your interest in the MyFantasyLeague.com Developers Program. The goal of this program is to provide the tools for our community to enhance the overall experience and enjoyment of the entire MFL user base. If you are a developer, these pages should provide you with all the information you need to create applications and other enhancements using the MFL platform.

New! For a list of changes made for the 2022 season, please check the [Release Notes](#) (last updated on July 25, 2022). 2020 Release Notes are [here](#) and 2019 Release Notes are [here](#)

Here is the overall information. You can get details on all the available calls on the [Request Reference Page](#), view [sample code](#) or [test the requests](#).

General Rules and Terms of Service

1. Access to this data is provided free to anyone to use in almost any way. However, the following uses are forbidden, whether deliberate or accidental:
 - Harvesting league and/or user data
 - Looking for loop holes or other ways to cheat or circumvent league rules
 - Overloading or disrupting the MFL service in any way that negatively impacts the normal operation of the site.
 - Collecting user information without their permission, attempting to make or making changes to someone's fantasy league/team without their permission.
 - Accessing the API via Javascript from web pages outside the myfantasyleague.com domain. Normal browser security will prevent this from working and we will not put your domain in our cross-domain file to allow it.
2. Access to this data is provided with the assumption that all developers will make their best effort to comply with the best practices listed here. Accidental or deliberate abuse of the service will not be tolerated.
3. Your application will perform much better if you cache data whenever possible. For example, our player database is only changed once a day, so your application should request that info no more than once a day and keep it for that long. You should do the same with league data that doesn't change often, like league rules and scoring systems.
4. This service is offered as is, without any guarantees expressed or implied. This service may be occasionally unavailable, or change or may be discontinued at any time, without warning.
5. If we determine that someone is abusing the system, whether deliberately or accidentally we may reject those requests and/or take whatever action we deemed necessary. Just

- because we allowed it in the past is no guarantee that we will allow it in the future.
6. We will do our best to answer your question regarding our API's. However we can't explain how to make API calls from your environment, or how to parse the results or any other details of how to use this data. Some help is available on the Sample Code link on the main API documentation page
 7. We can not and will not under any circumstance make raw NFL player stats available, as that's forbidden per our stats licensing agreement. Similarly, we can not make third party content (such as player news) available. If you are interested in getting raw NFL stats, here are some affordable options:
 - a. [FantasyData.com](#)
 - b. [Sportsradar](#)
 - c. [XML Team](#)
 8. These terms may be changed at any time. Use of the service constitutes agreement to these terms. We reserve the right to take whatever action we feel is warranted whenever anyone violates these terms.

Basic Information

In order to properly use the service you need to understand a few things about the requests you will be making, specially the URL format. All requests URL will be of the form:

protocol://host/year/command?args

For example, in the request:

<https://www44.myfantasyleague.com/2026/standings?L=80000&W=3>

the elements of the request are:

- *protocol*: https
- *host*: www44.myfantasyleague.com
- *year*: 2026
- *command*: standings
- *args*: L=80000&W=3

Each of these will be described below in more detail.

The **protocol** can be http or https. MFL started to support the https protocol in 2017 and at this time most requests can be made via either protocol. It is possible that in the next year we will require all API calls to be made in https so you may want to start planning for that.

The **host** is the server where the league you are accessing lives. Our hosts at this time are of the form wwwXX where XX is a 2-digit number. However that's likely to change in the near future. When you ask for a league information (via the *league* API) it will return the league's

host. Similarly, when getting all the leagues a user has (the *myleague* call) or using the league search functionality (the *leagueSearch* call), you will also receive the host information. You should save the league's host for period of time, but note that leagues can be moved between hosts. Moving leagues is unusual but does happen, specially if a server goes down or as we get ready for the start of the football season and need to balance load across hosts. Best practice is to get the data and cache it for one session. Note that if you send a request to the wrong host, the system will normally redirect the request to the proper server, but this operation may be delayed, or if the system is under too much stress, it may be denied. In addition, if the request does not take a league parameter (L=), it must be sent to the host **api**.

The **year** is fairly straight-forward. It refers to the season you are requesting data for. Note that January 2018 will still be part of the 2017 season, so this is not the current year. Also, most API functionality is only supported for the current year. We will not be adding features or fixing issues related to queries that refer to past seasons.

The **command** specifies what direction the data flows. Most requests will fall into two categories: those requesting data and those that will attempt to load data into the system. These requests use the commands *export* and *import* respectively. However, as listed on the [Request Reference Page](#), there are other commands that serve more specific purposes.

The **args** specifies the additional arguments for the request. The *export* and *import* commands require a request type, specified via the **TYPE** argument. This indicates the actual data you want back or want to update.

All requests that operate on a league require a league argument (**L**). The second most common one is the week argument (**W**), which is a number between 1 and 21. And finally the **JSON** argument should be set to 0 or 1 depending if you want the output in XML (default) or JSON format. Additional request parameters are listed in the [Request Reference Page](#).

User Agent And Client Registration

Starting in 2020 we will be monitoring and restricting usage of the API. This has become necessary because a number of bad actors are abusing the system. While most of these cases are likely just users not knowing any better, a few have been pretty clearly done with malicious intent.

The API will remain open as is always been. However, unregistered clients will be limited in the amount of requests they can make. The actual limits will depend on a number of factors and won't be constant. The limits will only apply on a per-IP address basis. So if your client is spread across many users, it won't be affected by this. Thus mobile apps and calls from within league pages should not be affected by this.

Registered clients will have higher limits, about 2.5X of the limits for un-registered clients. To

have your client receive the higher limits, you must:

1. Register your client via [API Client Registration Page](#).
2. Validate the client (this means that we will send a text to your cell phone number of record with a code on it that you will then enter on the site)
3. Pass the User-Agent you chose in the Client Registration on all API requests your client makes.

When you go over the limit, we will start throttling some requests from your client. The more you go over the limit, the more requests will be throttled. Throttled requests will return a 429 "Too Many Requests" HTTP status code. Here's some suggestions on what to do if you are hitting the limit:

1. Space your requests out. This is the most important thing you can do. Wait one second between making requests and you should be ok. We are not publishing what the limits are as they are not fixed and will vary depending on a lot of factors. For performance reasons we are not counting all requests, just a sample of them. So the limits won't ever be exact.
2. Cache the data so you don't have to request it as often. Some requests like players doesn't change often, so don't request it often.
3. Don't make unnecessary calls. Make calls depending on the point we are on the season. Asking for stats and results for week 5 during week 3 makes no sense and yet there are a lot of such calls being made. Also understand the league parameters. Don't ask for a league draft results when they do an auction.
4. Avoid making calls during high-traffic times. For instance during the NFL season we may lower the thresholds while games are going on. We may increase them during the late night/early morning hours.
5. If the requests are not league-specific, use `api.mylfantasyleague.com` as the host. That will spread out your requests across a number of servers. The limits are applied on a per server basis.
6. If a request fails, don't retry. That will make things worse. If you can, check for the 429 status and if you get it, cool things down a bit.

This is not a perfect system and if it turns out is not enough of a deterrent for these bad actors we will need to take stronger measures. We hope it doesn't come to that.

Other Arguments And Values

Players

All requests involving players identify players by their player id. Our player ids are 4 or 5 digit numbers, but are treated as strings internally. Ids under 1000 need a leading 0 (so the player id for the Baltimore Ravens defense is "0531", not "531"). If you treat them as integers you will run into issues.

Franchises

Some requests allow you to specify a franchise id. Franchise ids are always 4-digit numeric strings, with leading zeros, starting at "0001". Like player ids, franchise ids are strings, not numbers, so please make sure to treat them as such. In some cases the value "0000" may be passed in to indicate a commissioner operation.

Timestamps

All timestamps are specified using the [standard Unix time format](#). Your application needs to be able to convert times in this format to a user-readable form. The timestamps are always specified in the EST/EDT timezone.

Import Data

Some import requests require you to pass in an XML representation of the data being uploaded, via a field called 'DATA'. In most cases, you will need to properly encode this data so that special characters on it don't cause issues. It is also recommended that you use a POST request in these cases as that would make things easier in the long run. GET requests have length limits that can be exceeded very easily. With XML data is imperative that you get the formatting right as well as the various element and attribute names. The data being imported would always be in the same format as the corresponding export data.

Log In And Authorization

In order to access some league data, and to make any changes, your request must be properly authorized to access and/or change this data. Authorization is handled via cookies. Your request must pass in the cookie of the user who is performing the operation. Access will then be granted depending on the user's league privileges. Data from leagues that are marked as PRIVATE are no longer accessible by users not in the league.

To pass the proper user cookie, you must first programatically log the user in. You do this via the **login** API. The process is as follows:

- a. Prompt for the customer's username and password - you might want to do this similarly to how we do it on the [login](#) page.
- b. Programatically call the login API at: <https://api.myfantasyleague.com/2026/login?USERNAME=testuser&PASSWORD=testing&XML=1>, passing in a valid username and password. You must use HTTPS for this request to avoid exposing the user's credentials. For ease of illustrating this example, the request was shown using the GET method. For better security we recommend you call it using the POST method. We may stop supporting the GET method for login at any time.
- c. If valid information is passed into the login program, the response will include a `<status cookie_name="cookie_value"...>`.

- d. If invalid information is passed into the login program, it will respond with an <error... status message.
- e. The above cookie name/value pair should be passed back in as via a standard HTTP header [cookie](#) in the format: "Cookie: MFL_USER_ID=*cookie_value*" in all subsequent calls to the API.
- f. Note that the cookie value is a Base64 string. That means it may contain the special symbols '+', '/' and/or '='. Depending on your environment, you may need to explicitly URL-escape these symbols before passing them back to us (e.g. converting '=' to '%3D').

Note that some requests require commissioner access. For these requests you must pass the cookie of a user with commissioner access in that league. Requests that do not require commissioner access, when requested by a user who has commissioner access will be performed on the commissioner's franchise. Some requests (not all) can also be performed on behalf of another franchise by passing the FRANCHISE_ID parameter. If the commissioner does not have a franchise and no franchise id is given, it will return an error.

To log a user out, all the client needs to do is discard the cookie, or stop sending it in the API requests. There's no need to inform the server that this has occurred so there's no corresponding logout API call.

Because sometimes it is not possible to collect the user's credentials when making API requests we provide an alternate way to authorize requests. Note that for security reasons, this alternate method does not work for import requests, only export. It also does not work for requests that require Commissioner access (i.e. it's only valid for owners). To use it, you may pass to any request that requires both a league id and is access-restricted (i.e. requires a user cookie), the APIKEY parameter. The value for this parameter is user specific. It may be accessed via javascript on any page on the site when a user is logged in via the variable *apiKey*. Note that this API key is tied to a user/franchise/league combination and does not work outside that context. If you pass both a cookie and the APIKEY parameter, the APIKEY parameter will take precedence. This parameter is ignored on requests that don't require authorization.

You may also get your own API key and pass that on requests to your league. Since you are logged in on this league, your API key to access league 26696 is

"" (the key is everything inside the quotes not including the quotes). Do not share or make this info public as that may compromise the security of your league. But you may use this in your own personal programs for your own use. This key is good for the entire season but if you feel it is compromised you can contact us and we can invalidate it and assign you a new one.